

day 19

day 5 node

final project

Obsidian Notes: Node.js Day 5 Demo & Final Project Architecture

💡 الفكرة الأساسية للديمو النهائي

تم نقل مشروع الـ Backend لمستوى متقدم (Enterprise-Level Architecture) يحتوي على:

1. **Authentication:** JWT (JSON Web Tokens) التحقق من شخصية المستخدم وتأمين المسارات باستخدام.
2. **Authorization & RBAC:** التحكم في الصلاحيات بناءً على الأدوار (`superAdmin` , `admin` , `user`).
3. **Auto Seeding:** تلقائياً فور تشغيل السيرفر من خلال المتغيرات البيئية `superAdmin` إنشاء حساب.
4. **Middleware Pipeline:** تسلسل حماية محكم لكل طلب يتأكد من التوكين، الصلاحية، رفع الملفات، والتحقق من البيانات.

1. التشغيل والتأسيس التلقائي (`index.js` & `package.json`)

يحتوي ملف `index.js` على خطوة متقدمة جداً تسمى **Database Seeding**:

JavaScript

```
require("dotenv").config();
const express = require("express");
const app = express();
const userRouter = require("./routes/user");
const authRouter = require("./routes/auth");
const errorHandler = require("./middleware/error");
const notFoundHandler = require("./middleware/not-found");
const mongoose = require("mongoose");
const morgan = require("morgan");
const cors = require("cors");
const User = require("./model/user");

app.use(express.json());
app.use("/uploads", express.static("uploads"));
app.use(morgan("dev"));
app.use(cors());
```

```
// 🌐 Routers Integration
app.use(userRouter);
app.use(authRouter);

app.use(notFoundHandler);
app.use(errorHandler);

// 🚀 Start Server & Connect Database
app.listen(process.env.PORT || 3000, async () => {
  console.log(`Server is running on http://localhost:${process.env.PORT}`);
  await mongoose.connect(process.env.MONGODB_URI);
  console.log("DB Connected!");

  // 🔑 Auto-Seeding SuperAdmin
  // فحص ما إذا كان الحساب الرئيسي موجوداً، وإذا لم يوجد يتم إنشاؤه تلقائياً فور بدء التشغيل
  const superAdmin = await User.findOne({ email: process.env.SUPERADMIN_EMAIL });
  if (!superAdmin) {
    await User.create({
      name: process.env.SUPERADMIN_NAME,
      email: process.env.SUPERADMIN_EMAIL,
      password: process.env.SUPERADMIN_PASSWORD,
      role: "superAdmin",
    });
    console.log("SuperAdmin Created Successfully!");
  }
});
```

```

```
> [!TIP] 💡 **المكتبات الأساسية المستخدمة** (`package.json`)[cite: 21]:**
> * **`bcrypt`**: لعمل Hashing في داتا بيز [cite: 21].
> * **`jsonwebtoken`**: لإنشاء وفحص توكنات الـ [cite: 21].
> * **`joi`**: للتحقق من بيانات المدخلات (Validation)[cite: 21].
> * **`imagekit` & `multer`**: لرفع وإدارة الصور سحابياً ومحلياً [cite: 21].
```

---

## ## 🟡 2. طبقة التوثيق والصلاحيات (Authentication & Authorization)

**\*\*تعتمد هذه الطبقة على \*\*ميدل ويرين حاسمين**

```
(`middleware/auth.js`) التأمين والتحقق من الهوية**
وفكه بـ ، Header (Bearer Token) من الـ JWT يقوم هذا الميدل وير باستخراج الـ
`req.user` ثم جلب بيانات المستخدم وإلصاقها بـ ، `util.promisify(jwt.verify)`
```

```

```javascript
const util = require("util");
const jwt = require("jsonwebtoken");
const User = require("../model/user");
const AppError = require("../utils/AppError");

// تحويل jwt.verify من Callback إلى Promise مع استخدامه مع async/await
const jwtVerifyPromise = util.promisify(jwt.verify);

const auth = async (req, res, next) => {
  // 1. استخراج التوكين من Header
  const token = req.headers.authorization?.split(" ")[1];
  if (!token) {
    throw new AppError("Please login to access this route", 401);
  }

  // 2. فك التوكين والتأكد من صحته ورابط انتهائه
  const payload = await jwtVerifyPromise(token, process.env.TOKEN_SECRET_KEY);

  // 3. التثبت من وجود المستخدم صاحب التوكين في الداتابيز
  const user = await User.findById(payload._id);
  if (!user) {
    throw new AppError("User not found", 404);
  }

  // 4. استخدامها في الخطوات التالية Request إرفاق بيانات المستخدم بالـ
  req.user = user;
  next();
};

module.exports = auth;

```

ب) التحكم في الصلاحيات (middleware/restrictTo.js) [الجزء الشارح للمفهوم الناقص]

ميدل وير وظيفته التأكد من أن `req.user.role` يملك الصلاحية للوصول إلى هذا المسار (Role-Based Access Control):

JavaScript

```

const AppError = require("../utils/AppError");

const restrictTo = (...roles) => {
  return (req, res, next) => {
    // req.user تم تعيينه مسبقاً من auth middleware

```

```

    if (!roles.includes(req.user.role)) {
      throw new AppError("You do not have permission to perform this action",
403);
    }
    next();
  };
};

module.exports = restrictTo;

```

3. الموديل والتشفير الحساس (model/user.js) [الجزء الناقص المهم]

لكي تعمل عمليات التسجيل وال Login بكفاءة وأمان، يتم استخدام **Mongoose Hooks** لتشفير كلمة المرور تلقائياً باستعمال **bcrypt** قبل حفظها:

JavaScript

```

const mongoose = require("mongoose");
const bcrypt = require("bcrypt");

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: {
    type: String,
    enum: ["user", "admin", "superAdmin"],
    default: "user"
  },
  image: { type: String }
});

// 🔒 Pre-save Hook: تشفير الباسورد تلقائياً قبل الحفظ في MongoDB
userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// 🔑 Instance Method: مقارنة الباسورد المدخل مع الباسورد المشفر
userSchema.methods.comparePassword = async function (candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

```

```
const User = mongoose.model("User", userSchema);
module.exports = User;
```

4. تسلسل الأداء الفعلي للمسارات (& routers/user.js routers/auth.js)

تتجلى المعمارية الاحترافية في تسلسل الـ Middlewares داخل الـ Routers:

أ) مسارات التسجيل والـ (routers/auth.js) Login

:

JavaScript

```
const express = require("express");
const validate = require("../middleware/joi-validate");
const { signupSchema, loginSchema } = require("../utils/validate-schema/auth");
const { signup, login } = require("../controllers/auth");
const router = express.Router();

router.post("/signup", validate(signupSchema), signup); // تسجيل حساب جديد
router.post("/login", validate(loginSchema), login); // تسجيل الدخول والحصول
// على JWT Token

module.exports = router;
```[cite: 24]

المسارات المحمية (`routers/user.js`)[cite: 26]:
`POST /users`)[cite: 26]: شرايين الأمان في خطوة إضافة أدمن جديد
```javascript
router.post(
  "/users",
  auth, // 1 التأكد من تسجيل الدخول وفحص
  restrictTo("superAdmin", "admin"), // 2 التأكد من صلاحية المستخدم
  uploadOnMemory.single("img"), // 3 Memory استقبال الملف في الـ
  26]
  uplaodImageKit(false, "user-iti"), // 4 حفظ ImageKit رفع الصورة لـ
  URL[cite: 26]
  validate(createUserSchema), // 5 Joi بـ Body فحص مدخلات
  createAdmin // 6 التنفيذ النهائي وحفظ الحساب في
  DB[cite: 26]
);
```[cite: 26]
```

---

## 📄 ملخص الميدل ويرز وقواعد الأمان في مشروع الفاينل

الوظيفه والأهميه	Component
يتأكد من هوية صاحب Header، من الـ JWT يفك تشفير الـ	\*\*`auth` Middleware\*\*
`req.user`.	
أو مسارات Admins يمنع المستخدم من الوصول لإنشاء	\*\*`restrictTo` Middleware\*\*
(`403 Forbidden`).	
ببيانات ناقصة أو بصيغة Request يمنع وصول أي	\*\*`joi-validate` Middleware\*\*
(`400 Bad Request`).	
ويرفعها سحابياً لـ Buffer يستقبل الصورة كـ	\*\*`uploadImageKit` Middleware\*\*
`req.images`.	
افتراضي في الداتابيز عند `superAdmin` يضمن وجود حساب	\*\*`Auto-Seeding`\*\*
[cite: 20].	بداية التشغيل دون الحاجة لإنشائه يدوياً

---